Chapter 3, Numerical methods for initial value problems

As we have seen in Chapters 1 and 2, ODE-problems occur in numerous applications and therefore important for modeling processes whose evolution depends on one variable, usually time t or one spatial variable x. In this chapter we treat the initial value problem (IVP).

The general formulation of an IVP is:

$$\frac{d\mathbf{u}}{dt} = \mathbf{f}(t, \mathbf{u}), \quad \mathbf{u}(t_0) = \mathbf{u}_0, \quad t_0 < t \leq t_{end} \tag{3.1}$$

Often the solution depends on parameters, denoted by the vector $\mathbf{p}$, that occur in the right hand side function:

$$\frac{d\mathbf{u}}{dt} = \mathbf{f}(t, \mathbf{u}, \mathbf{p}), \quad \mathbf{u}(t_0) = \mathbf{u}_0, \quad t_0 < t \leq t_{end} \tag{3.2}$$

The initial values $\mathbf{u}_0$ can also be regarded as parameters in case the dependence of the initial values is studied: $\mathbf{u} = \mathbf{u}(t, \mathbf{p}, \mathbf{u}_0)$

Example 3.1 (Newton's law for a particle).

In exercise 2.1.4 the following IVP was introduced

$$m\ddot{y} = -mg - c\dot{y}|\dot{y}|, \quad y(0) = y_0, \quad \dot{y}(0) = v_0 \tag{3.3}$$

This ODE models a particle thrown vertically from the position y_0 and with initial velocity v_0. The particle is influenced by gravity and an air resistance force being proportional to the square of the velocity. When the solution is studied as a function of, e.g., c, we make a *parameter study* of the problem. The solution of this problem depends on t and the parameters m, c, g, y_0 and v_0, i.e. $y = y(t, m, c, g, y_0, v_0)$.

This example indicates that there are several questions that can be posed concerning the solution of an IVP:

- How does the solution behave on a time interval $[t_0, t_{end}]$ for a given $\mathbf{p}$?
- How does the solution depend on parameters $\mathbf{p}$ in the problem?
- Which are the critical points and what are the stability properties of the critical points (stability analysis)?
- How sensitive is the solution with respect to the parameters (sensitivity analysis)?

- When a parameter varies over an interval, does the solution change character for some value of the parameter in this interval (bifurcation analysis)?

- When measurements $(t_k, \mathbf{u}_k^*), k = 1, 2, \ldots, M$, of the process are given from experiments, how do we estimate unknown parameters in the model (parameter estimation)?

To be able to answer the questions, it is necessary to have an accurate, efficient and robust numerical solver for the IVP (3.1).

3.1 Graphical representation of solutions

To get a better understanding of the behaviour of solutions to ODE problems it is necessary to *visualize* the solutions graphically. This can be done in different ways:

- plot *trajectories*, i.e. $\mathbf{u}(t)$ is plotted as function of t in a coordinate system. A trajectory is appropriate when we want to study the time evolution of a process. Depending on the size of variables it should be judged what scales should be used for the different coordinate axis, linear or logarithmic.

- plot *phase portraits* showing two components of the solution, $u_i(t)$ and $u_j(t)$ with t as a parameter along the curve $(u_i(t), u_j(t))$ plotted in a coordinate system. A phase portrait is appropriate when one wants to study how solutions behave as function of initial values, especially in the neighbourhood of critical points. In 3D graphics a phase portrait with three components can be plotted.

There are other types of graphical representation, e.g., the *root locus plot*, often used in control theory to visualize the stability properties of an ODE-system, see example 2.7.

Example 3.2 (The kinetics of a biochemical reaction system).

The following reactions and the corresponding ODE-system was introduced in example 2.6:

$$E + S \Longleftrightarrow C \Longleftrightarrow E + P$$

The kinetics of the reaction system is modeled by the ODE-system (2.59). In this example the following values of the parameters are given: $k_1 = 10$, $k_2 = 0.1$, $k_3 = 1$, $k_4 = 10$. With the initial values $E_0 = 0.1$, $S_0 = 1$, $C_0 = 0$

and $P_0 = 0$ we get the following trajectories. Note that it is appropriate for this problem to use logarithmic scale on the t-axis (Figure 3.1).

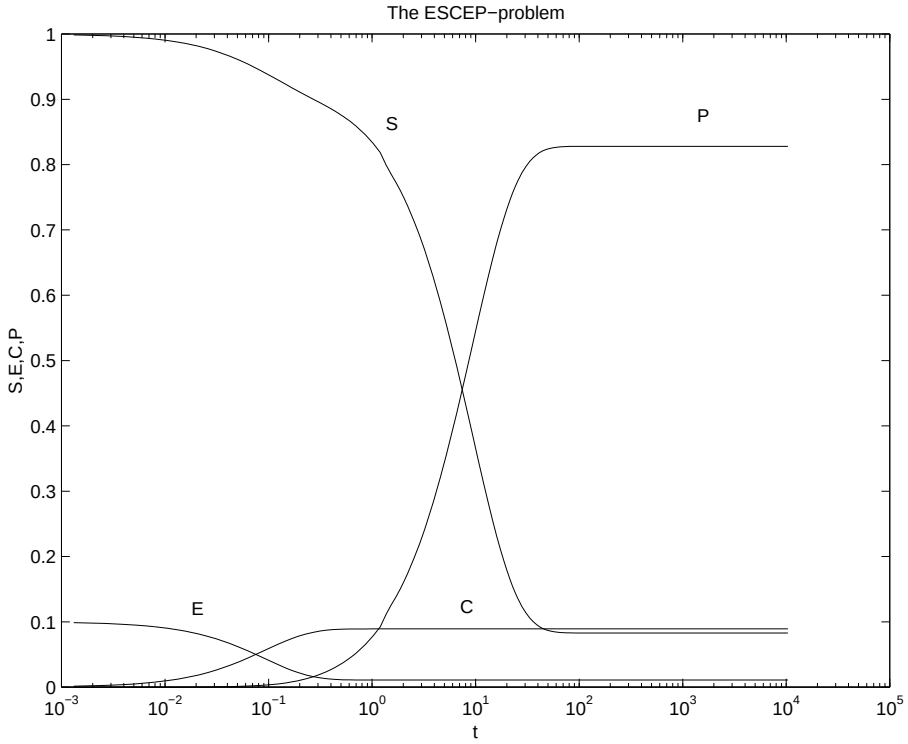


Figure 3.1: Example of solution trajectories

Example 3.3 (A problem with an oscillating solution: Van der Pol's equation).

Van der Pol's equation (see Exercise 2.3.5) occurs, e.g., in modeling of nonlinear electrical circuits:

$$\frac{d^2y}{dt^2} + \epsilon(y^2 - 1)\frac{dy}{dt} + y = 0, \quad y(0) = 1, \quad \frac{dy}{dt}(0) = 0 \tag{3.4}$$

The numerical solution of this ODE-problem is described in section 3.3. The results are preferably visualized as a phase portrait. If the parameter ϵ is given the values $\epsilon = 0.1, 1, 10, 100$ the following phase portraits are drawn in a *parameter study* (Figure 3.2). Obviously the solutions are oscillating. After an initial transient the phase portrait approaches a closed curve, a *limit cycle*. In the graph, the point o represents the initial value point.

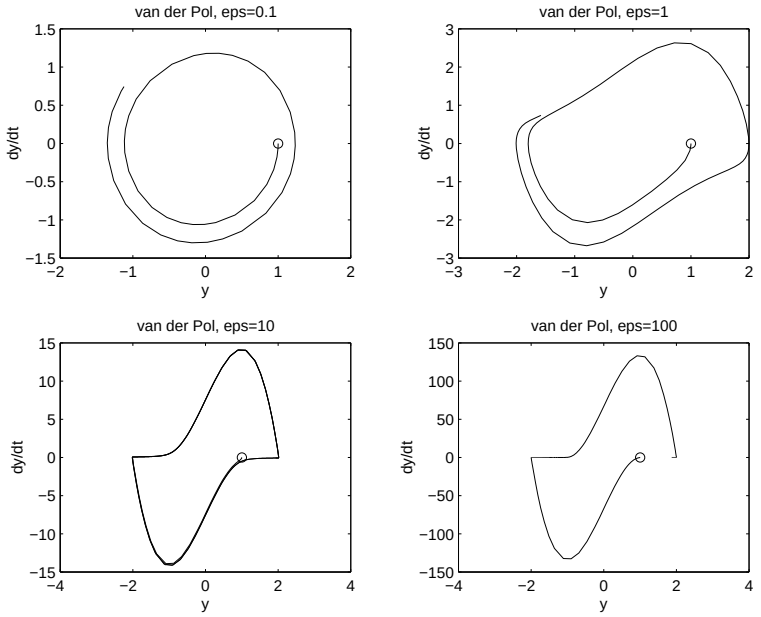


Figure 3.2: Example of phase portraits

3.2 Basic principles of numerical approximation of ODEs.

The solution of an ODE-problem (3.1) is a vector valued function $\mathbf{u}(t)$. An approximate numerical solution can be obtained with a *discretization* method, such as the *finite difference method* (FDM) or an *ansatz* method, e.g., the *finite element method* (FEM) or the *finite volume method* (FVM). In this chapter the FDM is presented.

With a FDM we obtain a sequence of points $(t_1, \mathbf{u}_1), (t_2, \mathbf{u}_2), \ldots, (t_N, \mathbf{u}_N)$. These points approximate the exact solution, i.e., $\mathbf{u}_k \approx \mathbf{u}(t_k)$. Examples of well-known FDMs are Euler's method and Runge-Kutta's method.

With an ansatz method a function $\mathbf{u}_h(x) \approx \mathbf{u}(x)$ is obtained. The approximating function has the form $\mathbf{u}_h(t) = \sum \alpha_j \varphi_j(x)$, where $\varphi_j(x)$ are given basis functions and α_j are coefficients chosen in such a way that the approximation is as accurate as possible in some sense. The finite element method (FEM) is an example of such a method, see sections 4.3, 6.5 and 7.4.

The FVM is a method for solving certain PDEs approximated by ODE systems. Similar to the FDM approximate values $\mathbf{u}_1, \mathbf{u}_2, \ldots, \mathbf{u}_N$ are calculated at discrete space points $x_1, x_2, \ldots, x_N$, see section 8.3.

When different methods for the IVP are discussed, we can without loss of generality restrict ourselves to the *scalar* case, i.e.,

$$\frac{du}{dt} = f(t, u), \quad u(0) = u_0, \quad 0 < t \leq t_{end} \tag{3.5}$$

The exact solution of this problem is denoted by $u(t)$. All numerical formulas in this chapter will be given for the scalar case, unless otherwise is stated.

3.3 Numerical solution of IVPs with Euler's method.

The basic idea in a FDM is that the derivative in (3.5) at time t_k is approximated by a *difference formula* using a few u-values $\dots u_{k-1}, u_k, u_{k+1} \dots$ in the neighbourhood of u_k. The difference h_k between two consecutive t-points, i.e. $h_k = t_{k+1} - t_k$, is called the *stepsize*. Most numerical methods use *variable stepsize*, i.e., h_k takes different values as the solution is computed along the t-axis. Methods where variable stepsize is implemented are also called *adaptive methods*.

However, *constant stepsize* h can also be used, especially when we want to have better control of the truncation errors (approximation errors) of the u_k-values. In that case the t_k-points are *equidistantly spaced* along the t-axis.

The discretized formulation of the ODE problem changes the differential equation into a *difference equation* and the numerical solution is computed as a sequence of u_k-values; the u_k-values are generated by a *recursion formula*. Linear difference equations are fundamental for understanding numerical stability of FDMs A short overview is given in Appendix A.2..

The derivative in (3.5) can be approximated by various difference formulas, e.g., the *Euler forward* (3.6) or the *Euler backward* (3.7) approximations:

$$\frac{du}{dt}(t_k) = \frac{u(t_{k+1}) - u(t_k)}{h} + O(h) \tag{3.6}$$

$$\frac{du}{dt}(t_k) = \frac{u(t_k) - u(t_{k-1})}{h} + O(h) \tag{3.7}$$

Both formulas are of first order, i.e. the truncation error is approximately proportional to h provided h is sufficiently small. Leonard Euler was a Swiss mathematician active in the 18th century and one of the most productive ever. He was one of the first to write a textbook in analysis: "Introductio in analysin infinitorum" from 1748.

Another possible approximation would be to use the *central difference* approximation formula, which is of second order:

$$\frac{du}{dt}(t_k) = \frac{u(t_{k+1}) - u(t_{k-1})}{2h} + O(h^2) \tag{3.8}$$

It may seem that the formula (3.8) is better to use instead of (3.6) and (3.7) since it is of second order and therefore more accurate. There are disad-

vantages, however, using (3.8) for the IVP, related to *numerical instability*, see section 3.4.2. For certain problems, however, (3.8) works well, see section 3.5.

Consider first the constant stepsize case. Applying (3.6) to the IVP (3.5) gives Euler's *explicit* method (also called Euler's forward method)

$$\begin{cases} u_k = u_{k-1} + hf(t_{k-1}, u_{k-1}), & u_0 = u(0) \\ t_k = t_{k-1} + h, & t_0 = 0, \quad k = 1, 2, \ldots, N \end{cases} \tag{3.9}$$

When (3.7) is used we get Euler's *implicit* method (also called Euler's backward method).

$$\begin{cases} u_k = u_{k-1} + hf(t_k, u_k), & u_0 = u(0) \\ t_k = t_{k-1} + h, & t_0 = 0, \quad k = 1, 2, \ldots, N \end{cases} \tag{3.10}$$

It can be shown that both methods give *first order accuracy* in the u_k-values, i.e., the truncation error, for IVPs also called the *global error*, e_k, fulfills

$$e_k = u(t_k) - u_k = O(h) \tag{3.11}$$

For both methods we see that when (t_{k-1}, u_{k-1}) is given as input, the next point (t_k, u_k) is computed as output, hence Euler's explicit and implicit methods are both *one-step* methods.

However, in explicit Euler, u_k is computed just by inserting (t_{k-1}, u_{k-1}) into the right hand side, while in implicit Euler u_k is computed by solving a nonlinear equation. For that reason we call (3.9) an *explicit* method while (3.10) is an *implicit* method.

Since an implicit method will be more complicated to compute one may immediately ask: Why use implicit Euler when the computational work is so much larger? The answer is that for certain problems called *stiff* problems, implicit Euler is much more advantegeous for *numerical stability* reasons. This will be shown in section 3.3.4.

The two Euler methods usually have too low order of accuracy to be of practical importance. However, they are very useful for demonstration of principles of the FDM when applied to an IVP. We will therefore show some basic properties using these two methods before we continue to higher order methods in section 3.4.

Exercise 3.3.1. Show that the difference approximations (3.6) and (3.7) both have first order accuracy.

Exercise 3.3.2. Show that the difference approximation (3.8) is of second order.

Exercise 3.3.3. Set up the recursion formula when Euler's explicit method (3.9) is applied to the *scalar model problem* for IVPs

$$\frac{du}{dt} = \lambda u, \quad u(0) = u_0$$

where λ is a constant complex parameter. Show that $u_k = (1 + h\lambda)^k u_0$.

Exercise 3.3.4. The following method is sometimes used for solving problems in Newtonian mechanics. For the ODE system

$$\frac{dv}{dt} = a(x, v), \qquad \frac{dx}{dt} = v$$

the Euler-Cromer method (see also Section 3.5) is defined as

$$v_{k+1} = v_k + ha(x_k, v_k), \qquad x_{k+1} = x_k + hv_{k+1}$$

What is the order of this method? Try an analytical approach or make a numerical experiment (see section 3.3.1).

3.3.1 Euler's explicit method: accuracy

We first show how explicit Euler is used to solve Van der Pol's equation (3.4)

$$\frac{d^2y}{dt^2} + \epsilon(y^2 - 1)\frac{dy}{dt} + y = 0, \quad y(0) = 1, \quad \frac{dy}{dt}(0) = 0$$

This second order ODE is written as a system of two first order ODEs by introducing a vector $\mathbf{u}$ with the components $u_1 = y, u_2 = dy/dt$:

$$\begin{cases} \frac{du_1}{dt} = u_2, \quad u_1(0) = 1 \\ \frac{du_2}{dt} = -\epsilon(u_1^2 - 1)u_2 - u_1, \quad u_2(0) = 0 \end{cases}$$

When the explicit Euler method is applied to this system we get the *recursion* formulas:

$$\begin{cases} u_{1,k} = u_{1,k-1} + hu_{2,k-1}, \quad u_{1,0} = 1 \\ u_{2,k} = u_{2,k-1} + h(-\epsilon(u_{1,k-1}^2 - 1)u_{2,k-1} - u_{1,k-1}), \quad u_{2,0} = 0 \\ t_k = t_{k-1} + h, \quad t_0 = 0 \end{cases}$$

Recall that $u_{1,k} \approx u_1(t_k) = y(t_k)$. How does the global error $e_{1,k} = u_1(t_k) - u_{1,k}$ depend on the stepsize h? The numerical experiment presented in the example below gives an answer to that question.

Example 3.4: Numerical experiment to determine the order of accuracy

N	h	$u_{1,N}$	$u_{1,N} - y(1)$
16	1/16	0.5231	0.0254
32	1/32	0.5100	0.0123
64	1/64	0.5037	0.0060
128	1/128	0.5006	0.0029
256	1/256	0.4991	0.0014
512	1/512	0.4984	0.0007
1024	1/1024	0.4980	0.0003
2048	1/2048	0.4978	0.0001

For Van der Pol's equation (3.4), let $\epsilon = 1$ and calculate $y(1)$ with explicit Euler. Discretize the t-axis on the interval $[0, 1]$ with constant stepsize h, using N steps. Table 3.1 shows the result obtained with the explicit Euler method.

Table 3.1: Numerical solution of Vnan der Pol's equation using explicit Euler

As we observe in the table, the convergence is very slow. When h is halved, the global error $u_{1,N} - y(1)$ is also halved, hence $e_{1,N} \approx Ch$

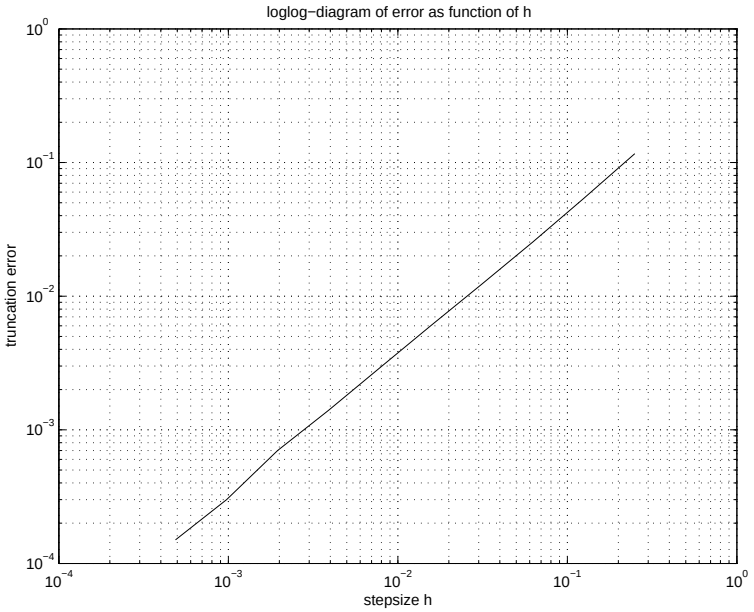


Figure 3.3: Loglog diagram showing the order of accuracy for explicit Euler

From a logarithmic plot, the order of accuracy can be estimated graphically. Taking the logarithm of the general error relation $e \approx Ch^p$ gives

$\log e \approx \log C + p \log h$, which is a straight line in a loglog diagram. The slope of that line is p. In Figure 3.3 the slope is 1, hence the order of explicit Euler is $p = 1$.

The error defined by (3.11) is called the *global error*. Another concept important for error estimation and variable stepsize control in adaptive methods is the *local error*, $l(t, h)$, which is defined as the residual obtained when the exact solution is inserted into the explicit Euler method

$$l(t_k, h) = u(t_k) - u(t_{k-1}) - hf(t_{k-1}, u(t_{k-1})) \tag{3.12a}$$

Hence the local error can be regarded as the truncation error committed in *one* step. With Taylor's expansion we see that

$$l(t_k, h) = \frac{h^2}{2}\frac{d^2u}{dt^2}(\tau_k), \quad t_k < \tau_k < t_{k-1} \tag{3.12b}$$

In Figure 3.4, it is shown how the global and local errors propagate when one step is taken from t_{k-1} to t_k.

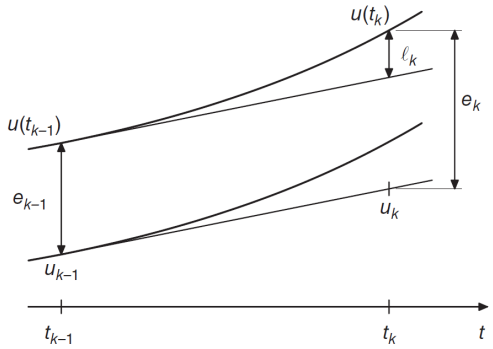


Figure 3.4: Graphical definition of the local and global errors

The global error e_k can be regarded as the accumulated effect of all local errors from t_1 up to t_k. However, it is not true in general that the global error is the sum of the local errors, as the following analysis shows:

Euler's explicit method can be written:

$$0 = -u_k + u_{k-1} + hf(t_{k-1}, u_{k-1})$$

If this equality is added to (3.12) we obtain, after using Taylor's expansion on the second argument u in $f(t, u)$, a global error recursion relation

$$e_k = e_{k-1} + h\frac{\partial f}{\partial u}(t_{k-1}, u_{k-1} + \theta(u(t_{k-1}) - u_{k-1}))e_{k-1} + l(t_k, h), \quad e_0 = 0 \tag{3.13}$$

where $0 < \theta < 1$. From this equality we see that only if $\partial f/\partial u \equiv 0$, i.e., if $f(t,u)$ does not depend on u, the global error is the sum of the local errors. If this is not the case assume that

$$\frac{\partial f}{\partial u} \leq \mu \Rightarrow 1 + h\frac{\partial f}{\partial u} \leq 1 + h\mu \leq e^{h\mu}$$

in the neighbourhood of the solution, where μ *can* be a negative number. Insert the expression (3.12b) into the global error formula (3.13) and we get

$$e_k \leq (1 + h\mu)e_{k-1} + \frac{h^2}{2}\frac{d^2u}{dt^2}(\tau_k)$$

With the assumption $h\mu > -1$ and

$$\max_{1\leq k\leq n} |\frac{d^2u}{dt^2}(\tau_k)| \leq M$$

we obtain the inequality

$$|e_k| \leq (1 + h\mu)|e_{k-1}| + \frac{h^2}{2}M$$

which gives the following estimate for the global error ($e_0 = 0$)

$$|e_k| \leq \frac{h^2}{2}M\frac{e^{t_k\mu} - 1}{h\mu} = O(h) \tag{3.14}$$

Hence the global error for Euler's explicit method is of first order. We see from (3.12b) and (3.14) that the local error is one order larger than the global error. In general this is true for most FDM. If the local error $l(t_k, h)$ is $O(h^{p+1})$, the global error e_k is $O(h^p)$.

3.3.2 Euler's explicit method: improving the accuracy

Instead of decreasing the stepsize h in explicit Euler to get better accuracy, the method can be improved by calculating the solution with two stepsizes, h and $h/2$, combined with *extrapolation*. Take a step from t_{k-1} to $t_k = t_{k-1} + h$ in the following way:

1) Take one Eulerstep of size h: $u_k^* = u_{k-1} + hf(t_{k-1}, u_{k-1})$

2) Take two Eulersteps of size $h/2$:

$$u_{k-\frac{1}{2}} = u_{k-1} + \frac{h}{2}f(t_{k-1}, u_{k-1}), \quad u_k^{**} = u_{k-\frac{1}{2}} + \frac{h}{2}f(t_{k-\frac{1}{2}}, u_{k-\frac{1}{2}})$$

3) The value u_k is obtained by extrapolation:

$$u_k = 2u_k^{**} - u_k^*$$

The method described in the steps 1 - 3 above can be shown to be a second order method. With a numerical experiment on Van der Pol's equation (3.4) this statement can be verified empirically (Table 3.2):

N	h	$u_{1,N}$	$u_{1,N} - y(1)$
4	1/4	0.4991	0.0015
8	1/8	0.4980	0.0004
16	1/16	0.4977	0.0001

Table 3.2: Numerical solution of Van der Pol's equation using Runge's 2nd order method

As is seen in Table (3.2) the errors decrease quadratically, i.e., $e_N = O(h^2)$. Hence the convergence of improved Euler is much faster than explicit Euler and therefore more efficient since fewer steps have to be taken to achieve a certain accuracy. The improved Euler method is equivalent to a *Runge-Kutta* method (more about these methods in section 3.4) known as Runge's second-order method:

$$k_1 = f(t_{k-1}, u_{k-1}),\ k_2 = f(t_{k-1} + h/2, u_{k-1} + hk_1/2),\ u_k = u_{k-1} + hk_2$$

Exercise 3.3.5. Show that improved Euler is a second order method. Hint: use the following error expansion valid for explicit Euler: $e_k = c_1 h + c_2 h^2 + c_3 h^3 + \ldots$ and extrapolation according to 3).

Exercise 3.3.6. Verify that improved Euler and Runge's second order method are the same one-step method.

Exercise 3.3.7. The following method, Heun's method, is another variant of explicit Euler.

$$u_k^* = u_{k-1} + hf(t_{k-1}, u_{k-1}), \quad u_k = u_{k-1} + \frac{h}{2}(f(t_{k-1}, u_{k-1}) + f(t_k, u_k^*))$$

Verify, by solving Van der Pol's equation numerically (write a program) that this method is of second order.

Exercise 3.3.8. Verify that the local error (3.12b) for explicit Euler is given by

$$l(t_k, h) = \frac{h^2}{2}\frac{d^2u}{dt^2}(t_k) + O(h^3)$$

Hence the local error is $O(h^2)$, while the global error is $O(h)$. Write a program to make a numerical experiment on the scalar test equation defined in exercise 3.3.3. Let $\lambda = -1$, $u_0 = 1$, $t_0 = 0$, $t_{end} = 1$ and $h = 0.05$. Compute the global and the local errors (neglect the $O(h^3)$-term) and write a table showing k, t_k, u_k, $l(t_k, h) = h^2u''(t_k)/2$, and $e_k = u(t_k) - u_k$.

3.3.3 Euler's explicit method: stability

In the previous part of this chapter it is shown how explicit Euler behaves when the stepsize h is small. But what happens if large steps are taken?

Recall first the example of instability in section 1.3, where the explicit Euler gives an erroneous result with increasing amplitudes in the numerical solution of the vibration equation, although the analytic solution has decreasing amplitudes. What is the reason for this difference in numerical and analytical behaviour?

Example 3.5: The following example from *chemical kinetics*, known as Robertson's problem, shows that explicit Euler gives wrong results unless the stepsize is small enough.

The following three irreversible reactions are given:

$$A \to B, \qquad B + C \to A + C \qquad 2B \to B + C$$

The following ODE-system can be formulated based on the mass action law:

$$\begin{cases} \frac{du_1}{dt} = -k_1u_1 + k_2u_2u_3, & u_1(0) = 1 \\ \frac{du_2}{dt} = k_1u_1 - k_2u_2u_3 - k_3u_2^2, & u_2(0) = 0 \\ \frac{du_3}{dt} = k_3u_2^2, & u_3(0) = 0 \end{cases} \tag{3.15}$$

where $k_1 = 0.04$, $k_2 = 10^4$, $k_3 = 3 \cdot 10^7$ are the rate constants of the reactions and u_1, u_2 u_3 are scaled concentrations of A, B and C.

If we use the explicit Euler method on this problem we get the following numerical result, shown in Table 3.3, for $\mathbf{u}(1)$ using different constant stepsizes h:

Table 3.3: Stability depending on the stepsize

N	h	$\mathbf{u}(1)$
10	0.1	NaN
100	0.01	NaN
1000	0.001	NaN
10^4	10^{-4}	ok

In the Table 3.3 NaN (Not a Number) means that the method has generated numbers that are too large to be stored in the computer memory, also called *overflow*. The result of the calculation is acceptable (but not very accurate) when $h = 10^{-4}$. However, the solution is interesting to study on the time interval $[0, 1000]$. Using the stepsize $h = 10^{-4}$ on the whole time interval means that we have to take 10^7 time steps, which is very inefficient.

The solution of the problem is shown in the graph in Figure 3.5 drawn in both linear and logarithmic scales:

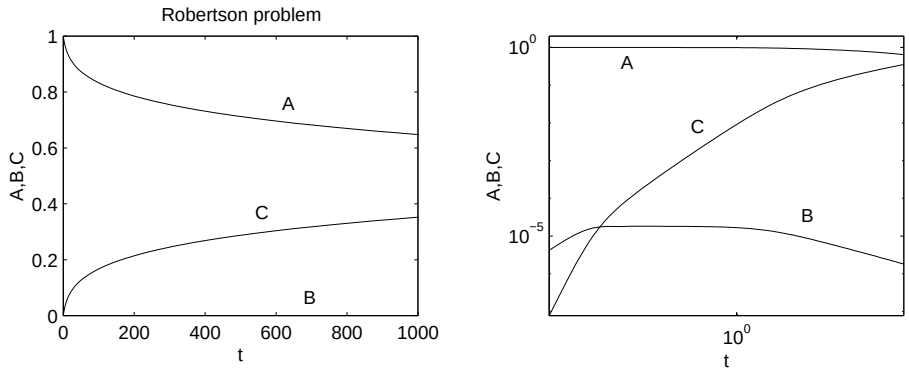


Figure 3.5: Solution of Robertson's problem, in linear and logarithmic diagrams

From the behaviour of the solution curves there seems to be no need for very small stepsizes; the solution is very smooth after a short transient phase in the time interval $(0, 10^{-2})$, where $u_2(t)$ passes a maximum.

Two relevant questions in connection to this problem are:

- Why are such small stepsizes needed when explicit Euler is used on this problem?
- Are there other methods better suited to solve this problem?

These questions and their answers are fundamental for understanding the *numerical stability* problems when solving both ODE and PDE problems by discretization.

To investigate the *numerical stability* of methods some results for LCC systems (see section 2.2) are needed.

We start with the *scalar model problem* introduced in exercise 3.3.3. Then we generalize to the LCC problem and finally to a nonlinear system. This is the same increase of complexity as in the analytic stability analysis in section 2.3.

$$\frac{du}{dt} = \lambda u, \quad \rightarrow \quad \frac{d\mathbf{u}}{dt} = A\mathbf{u}, \quad \rightarrow \quad \frac{d\mathbf{u}}{dt} = \mathbf{f}(\mathbf{u}) \tag{3.16}$$

Recall that the scalar model problem has the analytic solution $u(t) = e^{\lambda t}u_0$, which is stable for $t \geq 0$ if $Re(\lambda) \leq 0$.

Explicit Euler applied to the scalar model problem gives the following recursion formula (see exercise 3.3.3):

$$u_k = u_{k-1} + h\lambda u_{k-1} = (1 + h\lambda)u_{k-1} = (1 + h\lambda)^2 u_{k-2} = \ldots = (1 + h\lambda)^k u_0$$

The u_k-sequence is *numerically stable* (bounded) if

$$|1 + h\lambda| \leq 1 \tag{3.17}$$

If λ is real and negative, the inequality can be written:

$$-1 \leq 1 + h\lambda \leq 1 \Rightarrow h \leq \frac{2}{-\lambda} = \frac{2}{|\lambda|} = h_{max} \tag{3.18}$$

If λ is a complex number the geometrical correspondence to the inequality (3.17) is a disc in the complex $h\lambda$-plane with radius 1 and center in $h\lambda = -1$. We denote this region in the complex $h\lambda$-plane, by S_{EE}, the *stability region* for Euler's explicit formula (Figure 3.6).

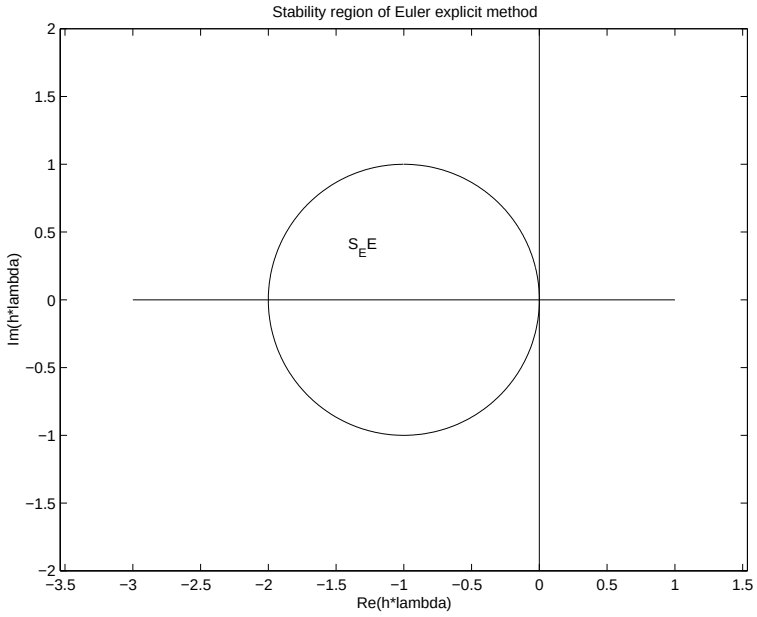


Figure 3.6: Stability region of Euler's explicit method

Hence Euler's explicit method gives stable solutions to the scalar model problem if

$$h\lambda \in S_{EE} \tag{3.19}$$

It is important at this stage to observe and understand the difference between analytic stability and numerical stability of the model problem $\dot{u} = \lambda u$.

- Analytic stability applies to the differential equation $\dot{u} = \lambda u$ and the stability criterion is formulated for λ only: $Re(\lambda) \leq 0$.

- Numerical stability applies to the difference equation obtained with the method, i.e. for explicit Euler $u_k = u_{k-1} + h\lambda u_{k-1}$ and the stability criterion is formulated for the product $h\lambda$ as follows: $h\lambda \in S_{EE}$. Observe that if $Re(\lambda) > 0$ the analytic solution of the model problem is unstable and the solution is unbounded as $t \to \infty$. In this case the explicit Euler method also gives unbounded numerical solutions since $h\lambda$ is situated in the right half plane.

For an LCC system of ODEs

$$\frac{d\mathbf{u}}{dt} = A\mathbf{u}, \quad \mathbf{u}(0) = \mathbf{u}_0$$

we know that the analytic solution is stable for $t \geq 0$ if $Re(\lambda_i) \leq 0$ (assuming there are no multiple eigenvalues with $Re(\lambda_i) = 0$).

Applying the explicit Euler method to this system gives as numerical solution a vector sequence $\mathbf{u}_k$:

$$\mathbf{u}_k = \mathbf{u}_{k-1} + hA\mathbf{u}_{k-1} = (I + hA)\mathbf{u}_{k-1} \tag{3.20}$$

The discussion of numerical stability is easier if we assume that the matrix A is diagonalizable, i.e.

$$A = S\Lambda S^{-1} \quad or \quad S^{-1}AS = \Lambda$$

where the columns of S are the eigenvectors of A and Λ is a diagonal matrix with the eigenvalues of A in the diagonal, see section 2.2.

Define another vector sequence $\mathbf{z}_k$ through the relation $\mathbf{u}_k = S\mathbf{z}_k$. Insert this into (3.20):

$$S\mathbf{z}_k = S\mathbf{z}_{k-1} + hAS\mathbf{z}_{k-1} \tag{3.21}$$

Multiply both sides by S^{-1}

$$\mathbf{z}_k = \mathbf{z}_{k-1} + hS^{-1}AS\mathbf{z}_{k-1} = (I + h\Lambda)\mathbf{z}_{k-1} \tag{3.22}$$

In the sequence (3.22) of vectors $\mathbf{z}_k$ the ith component $z_k^{(i)}$ is *uncoupled* from the other components since $I + h\Lambda$ is diagonal

$$z_k^{(i)} = (1 + h\lambda_i)z_{k-1}^{(i)} \tag{3.23}$$

Hence the stability criterion for the explicit Euler solution is the same as for the scalar model equation:

$$|1 + h\lambda_i| \leq 1, \qquad i = 1, 2, \ldots, n \tag{3.24}$$

The geometrical interpretation of these inequalities is that *all products* $h\lambda_i$ must be situated in the stability region S_{EE}, i.e.

$$h\lambda_i \in S_{EE}, \qquad i = 1, 2, \ldots, n \tag{3.25}$$

In the special case that all λ_i are real and $\lambda_n \leq \lambda_{n-1} \leq \ldots \leq \lambda_1 < 0$, we get the stability condition:

$$h \leq \frac{2}{|\lambda_n|} = h_{max} \tag{3.26}$$

i.e. the stepsize must be adjusted to the *fastest* timescale of the system. This may seem contradictory: The fastest component $e^{\lambda_n t}$ decreases very quickly

to zero and its value is very soon completely negligible. All the same this component puts the severest restriction to the stepsize!

For a nonlinear autonomous system of ODEs

$$\frac{d\mathbf{u}}{dt} = \mathbf{f}(\mathbf{u}), \qquad \mathbf{u}(0) = \mathbf{u}_0$$

the numerical stability analysis can be performed in a small neighbourhood of a point $(a, \mathbf{b})$ on a solution trajectory $(t, \mathbf{u}(t))$. We want to see how stepsizes can be chosen at this point. To apply the results from the previous LCC problem, approximate the nonlinear ODE system with an LCC system of ODEs. Let $\mathbf{u} = \mathbf{b} + \delta\mathbf{u}$, where $\delta\mathbf{u}(t)$ is a perturbation function. Insert this $\mathbf{u}$ into the ODE:

$$\frac{d\delta\mathbf{u}}{dt} = \mathbf{f}(\mathbf{b} + \delta\mathbf{u}) = \mathbf{f}(\mathbf{b}) + \frac{\partial\mathbf{f}}{\partial\mathbf{u}}(\mathbf{b})\delta\mathbf{u} + hot \tag{3.27}$$

Denote $\mathbf{f}(\mathbf{b})$ by $\mathbf{c}$ and the jacobian evaluated at $\mathbf{b}$ by J. If higher order terms are neglected the system of ODEs for the perturbation can be written

$$\frac{d\delta\mathbf{u}}{dt} = J\delta\mathbf{u} + \mathbf{c} \tag{3.28}$$

The constant $\mathbf{c}$ does not affect the stability analysis (see exercise 3.3.10). Therefore we have the (approximate) result that the explicit Euler method is stable if

$$h\lambda_i(J) \in S_{EE} \tag{3.29}$$

Let's test this result on Robertson's problem introduced in the beginning of this section. The jacobian of the system is

$$J = \begin{pmatrix} -k_1 & k_2u_3 & k_2u_2 \\ k_1 & -k_2u_3 - 2k_3u_2 & -k_2u_2 \\ 0 & 2k_3u_2 & 0 \end{pmatrix}$$

From the solution trajectory we choose the solution points $\mathbf{u}(1)$, $\mathbf{u}(10)$, $\mathbf{u}(100)$ and $\mathbf{u}(1000)$ and compute the eigenvalues of the jacobian in these points. The result is shown in Table 3.4.

Table 3.4: Eigenvalues of the jacobian of (3.15) along the solution trajectory

Since all eigenvalues are real and non-positive, the stability criterion (3.26) applies, so we conclude that a maximal stepsize h_{max} for Euler's explicit method would be $h_{max} = 2/3670 \approx 5 \cdot 10^{-4}$ for this problem in accordance

t	λ_1	λ_2	λ_3
1	0	-0.16	-1090
10	0	-0.06	-1210
100	0	-0.008	-1950
1000	0	-0.0004	-3670

with the numerical experiment in the beginning of this section. Using an adaptive variant of the method will not increase the efficiency since the stepsize must be kept very small on the whole time interval.

Robertson's problem is an example of a *stiff* ODE system. Such a system is characterized by the location in the complex plane of the eigenvalues $\lambda_i, i = 1, 2, \ldots, n$ of the jacobian. The system is stiff if

- λ_i are situated in the left half complex plane, i.e. $Re(\lambda_i) \leq 0$.
- $Re(\lambda_i)$ are of very different size, i.e. the system contains time constants of widely varying sizes.

The conclusion is that Euler's explicit method is not suited for stiff ODE problems.

Exercise 3.3.9. Show the instability of explicit Euler in example 1.1 by verifying that the values $h\lambda_i$ are not inside the stability region of explicit Euler for the stepsize used, $h = 0.1$. Which is the largest stepsize giving decreasing oscillations for this problem?

Exercise 3.3.10. Verify that the constant vector $\mathbf{c}$ in (3.28) does not affect the stability analysis, i.e. the stability results are the same as for the homogeneous ODE system. Hint: Let $\delta\mathbf{v} = \delta\mathbf{u} + J^{-1}\mathbf{c}$ and formulate (3.28) for $\delta\mathbf{v}$.

Exercise 3.3.11. Write a program to compute and plot the stability region S_H for Heun's method presented in exercise 3.3.7.

3.3.4 Euler's implicit method

Since Euler's explicit method is very inefficient on stiff problems, the question is if there are other methods that are better suited for these problems.

Apply Euler's implicit method on the scalar model problem:

$$u_k = u_{k-1} + h\lambda u_k \tag{3.30}$$

We solve this equation with respect to u_k and obtain

$$u_k = \frac{1}{1 - h\lambda} u_{k-1} \tag{3.31}$$

The sequence is stable if

$$|1 - h\lambda| \geq 1 \tag{3.32}$$

The stability region of the implicit Euler method is shown in the graph in Figure 3.7.

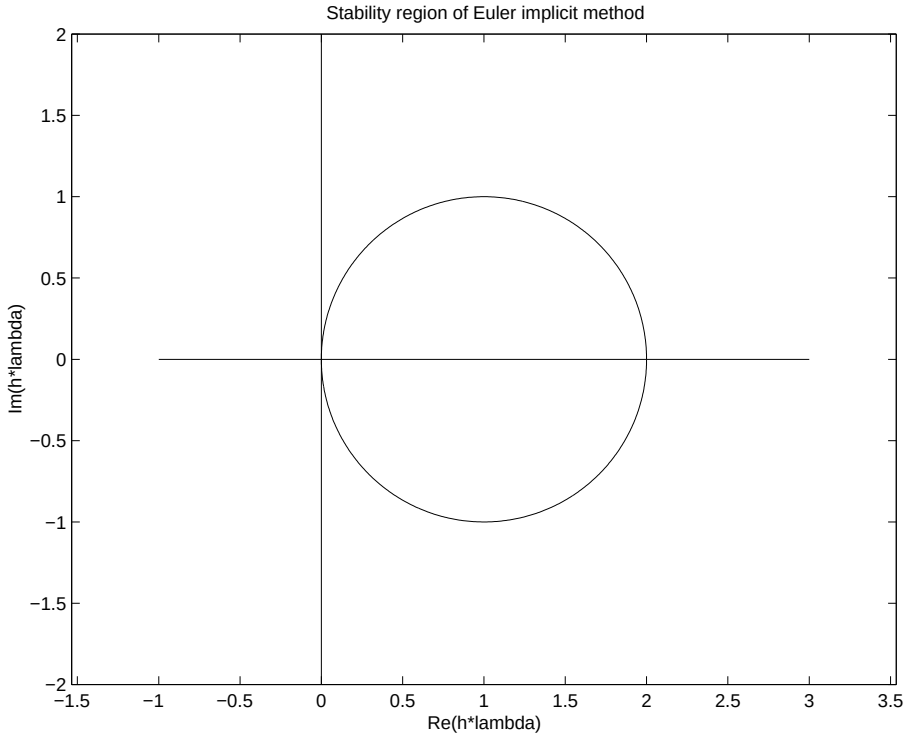


Figure 3.7: Stability region for Euler's implicit method

Hence the stability region S_{EI} consists of almost the whole complex plane; just the interior of the disc with radius 1 and center in $h\lambda = 1$ is *not* in the stability region. Therefore *the whole left half plane* belongs to the stability region. This method is appropiate for stiff systems since $h\lambda_i$ belongs to the stability region for all $h > 0$. Such a method is called an *A-stable* method. However, Euler's implicit method is only a first order method and will therefore be inefficient for *accuracy* reasons: the stepsize h must be small in order to meet a certain error tolerance. Also there is another price to pay. For a system of nonlinear ODEs the following nonlinear algebraic system of

equations must be solved with respect to $\mathbf{u}_k$ in *every* time step:

$$\mathbf{u}_k = \mathbf{u}_{k-1} + h\mathbf{f}(t_k, \mathbf{u}_k) \tag{3.33}$$

This can be done with Newton's method (see section A.1) which, however, consumes much computer time. An efficient method for stiff ODE-systems must be based on

- an implicit higher-order method
- variable stepsize technique
- use of the sparsity (see section A.5) of the jacobian

Exercise 3.3.12. Assume that $Re(\lambda) > 0$ in the scalar model problem, i.e. the analytical solution is unstable as $t \to \infty$. Find out if the numerical solution with implicit Euler is unstable or stable for a fixed λ-value as the stepsize h increases from zero to larger values.

3.3.5 The trapezoidal method

The trapezoidal method can be regarded as a symmetric combination of Euler's explicit and implicit methods

$$u_k = u_{k-1} + \frac{h}{2}(f(t_{k-1}, u_{k-1}) + f(t_k, u_k)) \tag{3.34}$$

The trapezoidal method is a one-step implicit method and has second order accuracy:

$$e_k = u(t_k) - u_k = O(h^2)$$

The stability region S_{TM} of the trapezoidal metthod is exactly the left half part of the complex plane (Figure 3.8)

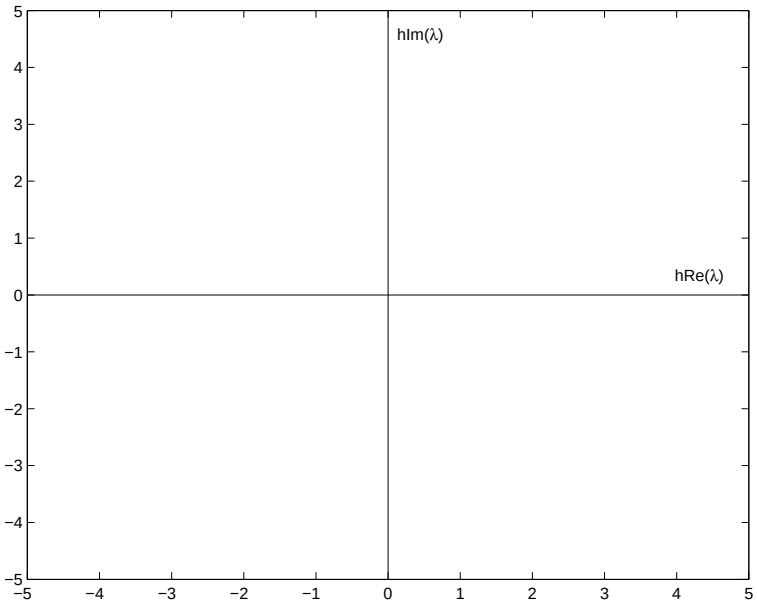


Figure 3.8: Stability region for the trapezoidal method

An advantage with the trapezoidal method is that the region of analytic stability coincides with S_{TM}. However, the solutions of the scalar model equation obtained with the this method will contain oscillations which are damped out very slowly when $\lambda << 0$. Therefore the method is not recommended for very stiff problems, use instead BDF-methods (see section 3.4.2).

Exercise 3.3.13. Write a program that visualizes the numerical solution of the trapezoidel method applied to the scalar model equation for some negative values of λ.

3.4 Higher order methods for the IVP.

3.4.1 Runge-Kutta methods

The improved Euler method (see section 3.3.2), also called Runge's second order method, is an example from the Runge-Kutta family of methods (RK methods). The most well-known member of this family is the *classical* RK4 method:

$$u_k = u_{k-1} + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \tag{3.35}$$
$$t_k = t_{k-1} + h, \qquad i = 1, 2, \ldots, N$$

where

$$k_1 = f(t_{k-1}, u_{k-1})$$
$$k_2 = f(t_{k-1} + h/2, u_{k-1} + hk_1/2)$$

$$k_3 = f(t_{k-1} + h/2, u_{k-1} + hk_2/2)$$
$$k_4 = f(t_{k-1} + h, u_{k-1} + hk_3)$$

Like Euler's method this is a one-step method, i.e., it is enough to know *one* point (t_{k-1}, u_{k-1}) in order to compute the next point (t_k, u_k). The term $(k_1 + 2k_2 + 2k_3 + k_4)/6$ in (3.35) can be regarded as a weighted mean value of slopes evaluated in four different points in order to obtain an accurate representation of the slope of the solution between (t_{k-1}, u_{k-1}) and (t_k, u_k). The RK4 method is a fourth order method, i.e., the truncation error e_k satisfies

$$e_k = u(t_k) - u_k = O(h^4) \tag{3.36}$$

Carl Runge and Wilhelm Kutta were German mathematicians active in the beginning of the 20th century.

Applying the RK4 method to Van der Pol's equation to compute $y(1)$, just as we did for Euler's method, results in Table 3.5:

N	h	$u_{1,N}$	$u_{1,N} - y(1)$
1	1	0.5052	0.0076
2	1/2	0.4981	0.0005
4	1/4	0.4976	$< 10^{-4}$
8	1/8	0.4976	$< 10^{-4}$

Table 3.5: Numerical solution of Van der Pol's equation using the RK4 method

Hence we obtain four digit accuracy for $N = 4$ which is much more efficient than the explicit Euler method.

Just like the explicit Euler method the RK4 method has a certain stability region, defined by applying the method to the scalar model equation. It is an algebraic exercise to show that the stability region for the RK4 method satisfies the inequality corresponding to the graph in Figure 3.9.

$$|1 + h\lambda + \frac{h^2\lambda^2}{2} + \frac{h^3\lambda^3}{6} + \frac{h^4\lambda^4}{24}| \leq 1 \tag{3.37}$$

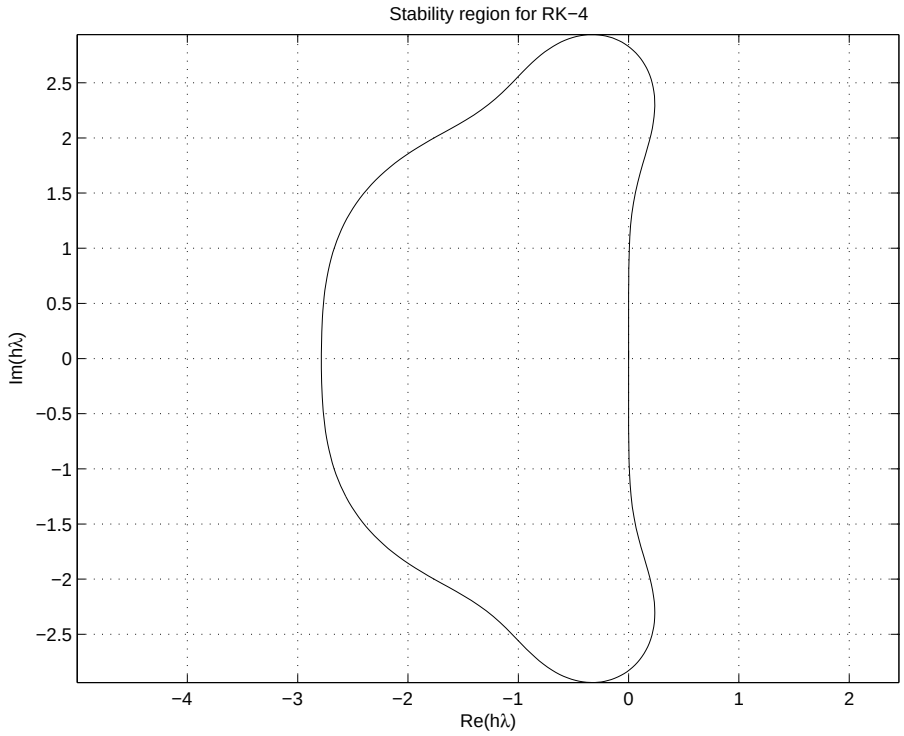


Figure 3.9: Stability region for the RK4 method

Since the stability region of RK4 is bounded and the order of accuracy is high the RK4 method is well-suited for *non-stiff* problems.

A more general formula for an explicit RK method is given by the following scheme of an *s-stage* RK method:

$$
\begin{aligned}
u_k &= u_{k-1} + h(b_1 k_1 + b_2 k_2 + \ldots + b_s k_s) \qquad (3.38)\\
t_k &= t_{k-1} + h, \qquad k = 1, 2, \ldots, N\\
k_1 &= f(x_{k-1}, u_{k-1})\\
k_2 &= f(x_{k-1} + c_2 h, u_{k-1} + h a_{21} k_1)\\
&\ldots\ldots\\
k_s &= f(x_{k-1} + c_s h, u_{k-1} + h(a_{s1} k_1 + a_{s2} k_2 + \ldots + a_{ss-1} k_{s-1}))
\end{aligned}
$$

In schematic form the formula can be described by a matrix tableau, the Butcher tableau, of the form

$$
\begin{pmatrix} \mathbf{c} & A \\ & \mathbf{b}^T \end{pmatrix} = \begin{pmatrix} 0 & & & & & \\ c_2 & a_{21} & & & & \\ \cdot & \cdot & \cdot & & & \\ c_s & a_{s1} & a_{s2} & \ldots & a_{ss-1} & \\ & b_1 & b_2 & \ldots & b_{s-1} & b_s \end{pmatrix} \qquad (3.39)
$$

The tableau (3.39) represents the formula for an explicit Runge-Kutta method. The matrix A is then lower triangular. Implicit Runge-Kutta methods can also be defined. In that case the matrix A in the tableau is full.

John Butcher is a numerical analyst from New Zeeland still active.

With this formalism the modified Euler method can be written as

$$\begin{pmatrix} 0 & & \\ 1/2 & 1/2 & \\ & 0 & 1 \end{pmatrix}$$

and the RK4 method as

$$\begin{pmatrix} 0 & & & & \\ 1/2 & 1/2 & & & \\ 1/2 & 0 & 1/2 & & \\ 1 & 0 & 0 & 1 & \\ & 1/6 & 1/3 & 1/3 & 1/6 \end{pmatrix}$$

Adaptive methods can be constructed from *embedded* RK-methods. An embedded RK-method uses the same **c**- and A-part of the matrix tableau, but consists of two different **b**-vectors. The first one, denoted by **b** (giving a method of order p), is used to compute the next value u_k. The second one denoted $\hat{\mathbf{b}}$ (giving a method of order $p+1$), computes a $\hat{u}_k$ used just for error estimation:

$$\begin{pmatrix} \mathbf{c} & A \\ & \mathbf{b}^T \\ & \hat{\mathbf{b}}^T \end{pmatrix} = \begin{pmatrix} 0 & & & & & \\ c_2 & a_{21} & & & & \\ . & . & . & & & \\ c_s & a_{s1} & a_{s2} & \dots & a_{ss-1} & \\ & b_1 & b_2 & \dots & b_{s-1} & b_s \\ & \hat{b}_1 & \hat{b}_2 & \dots & \hat{b}_{s-1} & \hat{b}_s \end{pmatrix} \tag{3.40}$$

Assume that the local error (see section 3.3.1) of the method corresponding to **b** satisfies

$$l(t_k, h) \approx Ch^{p+1} \tag{3.41}$$

Assume also that $\hat{u}_k$ is much more accurate than u_k, so that $\delta = |\hat{u}_k - u_k|$ is an estimate of $l(t_k, h)$ for the step taken from t_{k-1} to t_k.

If *tol* is a given error tolerance value for the absolute error we can use the following stepsize strategy when taking a step h from t_{k-1} to t_k:

Algorithm 3.1 (Stepsize control)

1. *accept* the step (u_k is accepted) if $\delta \leq tol$

   The new step h_{next} should satisfy $tol = Ch_{next}^{p+1}$, hence $h_{next} = h(\frac{tol}{\delta})^{1/(p+1)}$.

2. *reject* the step (u_k is rejected) if $\delta > tol$

   Start again from t_{k-1} with the new stepsize $h_{new} = h/2$, return to 1.

One well-known embedded Runge-Kutta method is the Bogacki-Shampine order 2/3 pair, used in e.g. ode23 in the MATLAB(R)-library

$$\begin{pmatrix} 0 & & & & \\ 1/2 & 1/2 & & & \\ 3/4 & 0 & 3/4 & & \\ 1 & 2/9 & 3/9 & 4/9 & 0 \\ & 2/9 & 3/9 & 4/9 & 0 \\ & 7/24 & 6/24 & 8/24 & 3/24 \end{pmatrix}$$

Here $\hat{\mathbf{b}}$ corresponds to a second order method, while $\mathbf{b}$ is of order three.

Exercise 3.4.1. Formulate the RK-method associated with the following tableau

$$\begin{pmatrix} 0 & & & \\ 1 & 1 & & \\ 1/2 & 1/4 & 1/4 & \\ & 1/6 & 1/6 & 4/6 \\ & 1/2 & 1/2 & 0 \end{pmatrix}$$

Apply this method to the test problem $\dot{u} = \lambda u$ and deduce from that result the order of the method giving the solution values and also the order of the method used for error estimation.

Exercise 3.4.2. Show the inequality (3.37) for the stability of the RK4 method.

Exercise 3.4.3. Write a program for plotting the graph of the stability area of the RK4 method.

Exercise 3.4.4. Write a program for simulation of the particle motion problem described in example 2.3. Use the RK4-method with constant stepsize. The following values of the model parameters can be used: $g = 10\,[m/s^2]$, $\|\mathbf{v}_0\| = 10\,[m/s]$, $y0 = 2\,[m]$, $c = 0.01\,[N \cdot s^2/m]$, $m = 1\,[kg]$, $\alpha = 20^o$, and $\alpha = 60^o$. Make a plot of the two particle trajectories showing the motion from the initial point to the point where the particle hits the ground.

3.4.2 Linear Multistep methods

In linear multistep methods information is used not only from the previous step (t_{k-1}, u_{k-1}) but also from earlier steps (t_{k-2}, u_{k-2}), (t_{k-3}, u_{k-3}), ..., (t_{k-p}, u_{k-p}).

A simple example of a *two-step method* is the *explicit midpoint method*, also known as the *leap-frog method* based on the central difference approximation (3.5)

$$\begin{cases} u_k = u_{k-2} + 2hf(t_{k-1}, u_{k-1}) \\ t_k = t_{k-1} + h, \quad k = 2, 3, ..., N \end{cases} \tag{3.42}$$

Inserting $k = 2$ in the formula we see that the method is not self-starting, i.e. apart from u_0, the initial value, we also need a value of u_1. This could be obtained e.g. by using the explicit Euler method, i.e. $u_1 = u_0 + hf(t_0, u_0)$.

The midpoint method is a second order method, but unfortunately it is not suited for most kinds of IVPs (see, however, section 3.5) which the following example shows:

Example 3.6 Consider the explicit midpoint method applied to the scalar model problem. We get the difference equation:

$$u_k = u_{k-2} + 2h\lambda u_{k-1}, \qquad u_0 = 1, \quad u_1 = 1 + h\lambda$$

The results for $h = 0.1$,$\lambda = -1$ and $\lambda = -2$ are shown in the graphs in Figure 3.10.

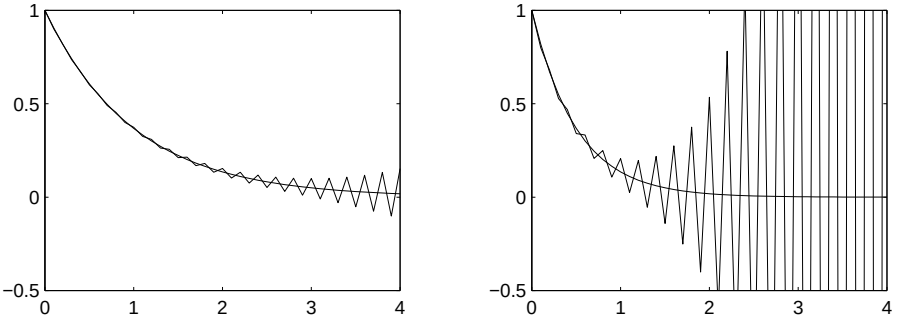


Figure 3.10: Numerical instability for the leap-frog method

The analytic solution is the smooth curve and the numerical solution is the oscillating curve with increasing amplitude. This method is unstable regardless of the stepsize h. It does not help to decrease the stepsize; the oscillations will occur sooner or later. Hence the explicit midpoint method is not suited for solving IVP in general, but can be designed to work for

mechanical problems, see section 3.5. The stability region is derived in section A.2.

The general definition of a linear p-step method is

$$u_k = \alpha_1 u_{k-1} + \alpha_2 u_{k-2} \ldots \alpha_p u_{k-p} + h(\beta_0 f_k + \beta_1 f_{k-1} + \ldots \beta_p f_{k-p}) \quad (3.43)$$

where $f_k = f(t_k, u_k)$. To start such a method we need apart from u_0 also u_1, u_2, ... u_{p-1}. These values can be calculated with some one-step method, e.g., the RK4 method.

If $\beta_0 = 0$ the method is *explicit*, if $\beta_0 \neq 0$ the method is *implicit*. Some well-known classical multistep methods are, e.g.,

- Adams-Bashforth (explicit):

$$u_k = u_{k-1} + h(f_{k-1} + \frac{1}{2}\nabla f_{k-1} + \frac{5}{12}\nabla^2 f_{k-1} + \ldots) \quad (3.44)$$

- Adams-Moulton (implicit)

$$u_k = u_{k-1} + h(f_k + \frac{1}{2}\nabla f_k - \frac{1}{12}\nabla^2 f_k + \ldots) \quad (3.45)$$

- Backward Differentiation Formulas (also called BDF-p or Gear's method of order p and well suited for stiff IVPs)

$$\nabla u_k + \frac{1}{2}\nabla^2 u_k + \ldots + \frac{1}{p}\nabla^p u_k = h f_k, \quad p \leq 6 \quad (3.46)$$

  where

$$\begin{aligned} \nabla f_k &= f_k - f_{k-1} \\ \nabla^2 f_k &= f_k - 2f_{k-1} + f_{k-2} \\ \nabla^3 f_k &= f_k - 3f_{k-1} + 3f_{k-2} - f_{k-3} \end{aligned} \quad (3.47)$$

J.Adams was a British mathematician active in the middle of the 19th century. F.Bashforth and F.Moulton were scientists active around 1900 and W.Gear is an Americal numerical analyst who published his method around 1970.

Automatic stepsize control can be designed in a way similar to RK methods. For linear multistep methods the *local error* $l(t_k, h)$ (see section 3.3.1) is estimated as follows:

$$l(t_k, h) = C u^{(p+1)}(t_k) h^{p+1} \quad (3.48)$$

where C is a known constant depending on the method. The derivative can be estimated from some difference approximation, see appendix A3. As an example for Adams-Bashforth method, $p = 1$, (which is in fact explicit Euler) the local error is estimated from

$$l(t_k, h) = \frac{1}{2}\frac{\nabla^2 u_k}{h^2}h^2 = \frac{\nabla^2 u_k}{2} \tag{3.49}$$

and the algorithm for stepsize control now follows algorithm 3.1.

Exercise 3.4.5. Present in the form (3.43) the first and second order methods of Adams-Bashforth, Adams-Moulton and Gear.

Exercise 3.4.6. Compute the stability region of BDF-2.

Exercise 3.4.6. Compute the solution of the ESCEP-problem described in example 2.6. Use BDF-2 with stepsize $h = 0.01$. (It is of course more effective to use variable stepsize).

3.5 Special methods for special problems

In many physical and chemical problems it is important in numerical simulations to preserve certain invariants, such as the total energy, the total mass and/or the positivity of the solution components. Many important classes of models possess invariants which are linear and quadratic expressions in the solution components $u_1, u_2, \ldots u_n$. In numerical calculations we cannot expect the invariants to be preserved exactly but should be preserved to within rounding errors.

3.5.1 Preserving linear and quadratic invariants

Linear invariants occurring, e.g., in the conservation of mass of a system, are preserved by most methods solving the IVP. For an ODE system, assume that a number of linear relations are time invariant

$$\frac{d\mathbf{u}}{dt} = \mathbf{f}(t, \mathbf{u}),\ \mathbf{u}(0) = \mathbf{u}_0, \qquad B\mathbf{u}(t) = B\mathbf{u}_0$$

where B is an $m \times n$, $m < n$ matrix. By differentiating the linear relation with respect to t we obtain

$$B\frac{d\mathbf{u}}{dt} = B\mathbf{f}(t, \mathbf{u}) = 0$$

When using e.g. explicit Euler we get after multiplication by B

$$B\mathbf{u}_k = B\mathbf{u}_{k-1} + hB\mathbf{f}(t_{k-1}, \mathbf{u}_{k-1}) = B\mathbf{u}_{k-1} = \dots B\mathbf{u}_1 = B\mathbf{u}_0$$

Hence the linear relations are preserved along the approximate solution trajectory $\mathbf{u}_1, \mathbf{u}_2, \dots \mathbf{u}_n$. It is easy to conclude the same property for e.g. classical Runge-Kutta methods and linear mulitstep methods. Observe that even if the components of the solution vector $\mathbf{u}$ are computed with a global error of say 10^{-4} the computed linear invariants will hold an accuracy on rounding error level, typically 10^{-15}.

Example 3.7 In the biochemical system presented in example 2.6 the matrix S in the right hand side of the ODE system (2.59) has rank 2, hence there are two linear relations between the rows of S, see exercise 2.5.7.

$$B\mathbf{f}(\mathbf{c}) = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} -1 & 1 & 1 & -1 \\ -1 & 1 & 0 & 0 \\ 1 & -1 & -1 & 1 \\ 0 & 0 & 1 & -1 \end{pmatrix} \mathbf{r}(\mathbf{c}) = 0$$

for all vectors $\mathbf{c}$, hence $B\mathbf{c} = B\mathbf{c}_0$.

In general quadratic invariants are not preserved, but there is one method, a variant of the trapezoidal method, called the *implicit midpoint method*

$$\mathbf{u}_k = \mathbf{u}_{k-1} + h\mathbf{f}(t_{k-1} + \frac{h}{2}, \frac{\mathbf{u}_{k-1} + \mathbf{u}_k}{2}) \tag{3.50}$$

that preserves the relation $\mathbf{u}^T C\mathbf{u} = \mathbf{u}_0^T C\mathbf{u}_0$, where C is a symmetric matrix. By time differentiation of the quadratic relation and dividing by 2 we get

$$\mathbf{u}^T C\frac{d\mathbf{u}}{dt} = \mathbf{u}^T C\mathbf{f}(t, \mathbf{u}) = 0$$

Multiply (3.54) first by $\mathbf{u}_{k-1}^T C$, then by $\mathbf{u}_k^T C$, add these two equalities and you find that $\mathbf{u}_k^T C\mathbf{u}_k = \mathbf{u}_{k-1}^T C\mathbf{u}_{k-1}$, hence the quadratic relation is preserved.

Exercise 3.5.1. Given the parameter representation of a circle of radius 1

$$x = cos(\varphi), \qquad y = sin(\varphi), \quad 0 \le \varphi \le 2\pi$$

and the corresponding ODE system where $u_1 = x$ and $u_2 = y$.

$$\frac{d\mathbf{u}}{d\varphi} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \mathbf{u}, \quad \mathbf{u}(0) = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

Write a program showing graphs of the phase portraits (u_1, u_2) obtained with the a) explicit Euler, b)implicit Euler and c) implicit midpoint method. Conclude that explicit Euler gives a growing spiral, implicit Euler gives a shrinking spiral and the implicit midpoint method gives a circle. Try also to verify this by a mathematical proofs.

3.5.2 Preserving positivity of the numerical solution

In the numerical simulation of physical and chemical systems there is a demand for positivity for certain solution components. A concentration or a pressure, e.g., can never take negative values. Apart from being unphysical, negativity may also make the ODE system unstable and totally ruin the whole solution. A simple remedy would be, e.g., to set a negative concentration component to zero, which, however, will destroy the mass balances (linear invariants) of the system.

Preserving numerical positivity when solving an ODE system with solution components $u_i(t) > 0$ as long as $u_i(0) \geq 0$ is harder. One way can be to utilize the special form of the right hand side of the ODE system. As an example, take the system (2.49) being a model for chemical kinetics. We can write this system in the alternative form

$$\frac{d\mathbf{c}}{dt} = K(\mathbf{c})\mathbf{c}, \quad \mathbf{c}(0) = \mathbf{c}_0 \tag{3.51}$$

where $K(\mathbf{c})$ is an $N \times N$ matrix. Using implicit Euler on this usually stiff ODE system gives the algebraic equation system

$$\mathbf{c}_k = \mathbf{c}_{k-1} + hK(\mathbf{c}_k)\mathbf{c}_k \rightarrow (I - hK(\mathbf{c}_k))\mathbf{c}_k = \mathbf{c}_{k-1}$$

For the special matrices $K(\mathbf{c})$ occurring in chemical kinetics, it can be shown that if $\mathbf{c}_{k-1} > 0$ (all components of $\mathbf{c}_{k-1}$ are positive), the inverse of $I - hK(\mathbf{c}_{k-1})$ has all elements positive, thus $\mathbf{c}_k > 0$, and positivity will be preserved if the initial value $\mathbf{c}_0$ is non-negative. Now, if the nonlinear equation system is solved with an iteration method of type

$$(I - hK(\mathbf{c}_k^{(i)}))\mathbf{c}_k^{(i+1)} = \mathbf{c}_{k-1}$$

then positivity is preserved. On the other hand that iteration scheme does not give quadratic convergence. Newton's method (see Appendix A.1) does but will not always produce positive iterates, at least not for chemical kinetics ODEs.

3.5.3 Methods for Newton's equations of motion

For conservative mechanical systems the total energy is preserved. One very important application is *molecular dynamics*, where the physical movements of interacting molecules are simulated by numerically solving Newton's equation of motion (2.43). The Hamiltonian (2.46) for such a system of n particles can be written as the sum of the kinetic energy T and the potential energy U

$$H = T + U = \sum_{i=1}^{n} \frac{\|\mathbf{p}_i\|^2}{2m_i} - \sum_{j=1, i\neq j}^{n} \frac{\gamma m_i m_j}{\|\mathbf{r}_i - \mathbf{r}_j\|} \tag{3.52}$$

Newton's equation will be

$$m_i \ddot{\mathbf{r}}_i = - \sum_{j=1, i\neq j} \frac{\gamma m_i m_j (\mathbf{r}_i - \mathbf{r}_j)}{\|\mathbf{r}_i - \mathbf{r}_j\|^3}, \quad i = 1, 2, \ldots, n$$

Another important application of a mechanical system is a system of masses and springs introduced in (2.44) (with $D_\nu = 0$ and $\mathbf{F} = 0$) and for which Newton's equations take the form

$$M\ddot{\mathbf{x}} + K\mathbf{x} = 0, \quad \mathbf{x}(0) = \mathbf{x}_0$$

For this ODE system the hamiltonian H (quadratic form) is invariant

$$H = \frac{1}{2}\dot{\mathbf{x}}^T M \dot{\mathbf{x}} + \frac{1}{2}\mathbf{x}^T K \mathbf{x} \tag{3.53}$$

For Newton's equation of motion special numerical methods have been developed and much used, e.g. the leap-frog method and Verlet's method, which are both of second order accuracy, but do not belong to the Runge-Kutta or linear multistep methods. One reason for using special methods for Newton's equation is that Runge-Kutta and linear multistep methods do not preserve the total energy of a conservative mechanical system but allow the energy to drift away from its constant value as time increases. The Verlet method has its name from the French physicist Loup Verlet, who rediscovered the method for computer simulation of molecular dynamics in the 1960s.

If we write Newton's equation of motion on scalar form as a first order system

$$\dot{r} = v, \quad r(0) = r_0 \qquad \ddot{r} = a(r), \quad v(0) = v_0 \tag{3.54}$$

where $a(r)$ depends only on the position r, Verlet's method for the position and the velocity is

$$\begin{cases} v_k = (r_{k+1} - r_{k-1})/2h \\ r_{k+1} = 2r_k - r_{k-1} + h^2 a(r_k) \end{cases} \tag{3.55}$$

As is seen from (3.55) the derivative approximations are based on the symmetric central difference formulas (3.8) and (4.24). The method has second order accuracy. It is also explicit and needs only one evaluation of the right hand side function $a(r)$ per time step. If just the positions $r_0, r_1, \ldots r_k, ..$ are wanted only the second of the two formulas are needed. This method also goes under the name Störmer's method, after the Norwegian mathematician Carl Störmer, active in the first part of the 20th century.

Verlet's method is not self-starting meaning that the initial conditions $r_0 = r(0)$ and $v_0 = v(0)$ are not enough to start the recursion. Here r_1 is also needed, and can be computed from some approximation of the first step taken, e.g.,

$$r_1 = r_0 + hv_0 + \frac{h^2}{2}a(r_0), \tag{3.56}$$

The order of computation will then be r_0, v_0, r_1, r_2, v_1,r_3,v_2, etc.

The leap-frog method is based on the following representation of Newton's equations of motion: $\dot{v} = a(r), \quad v(0) = v_0, \qquad \dot{r} = v, \quad r(0) = r_0$

$$\begin{cases} v_{k+1} = v_{k-1} + 2ha(r_k) \\ r_{k+2} = r_k + 2hv_{k+1} \end{cases} \tag{3.57a}$$

which can also be written with midpoint values

$$\begin{cases} v_{k+1/2} = v_{k-1/2} + ha(r_k) \\ r_{k+1} = r_k + hv_{k+1/2} \end{cases} \tag{3.57b}$$

This method is also based on the symmetric approximation of the derivatives. It has a strong resemblance with the Euler-Cromer method (see Exercise 3.3.4) which, however, is based on the Euler forward approximation giving only first order accuracy, while the leap-frog method is of second order. In the beginning of section 3.4.2, the leap-frog method was tested on the scalar model problem $\dot{u} = \lambda u$ and found to be useless when, e.g., $\lambda = -1$. However, for hamiltonian systems in mechanics it turns out to be both effective and reliable, just like Verlet's method. Here we also need a way to start the method, e.g., with (3.56). The order of computation will be r_0, v_0, r_1, v_2, r_3, v_4, r_5, etc. From this sequence and Figure 3.11 we see that velocities v are evaluated at a grid *staggered* with respect to the positions r. In general a staggered grid method means that different variables are computed on different grids, see further section 8.2.7.

Figure 3.11: Steps taken by different variables on a staggered grid

3.6 The variational equation and parameter fitting in IVPs

The variational equation (VE) was introduced in chapter 2 as a tool for analytic *stability analysis* of solution curves and critical points. The aim was to see if a perturbed curve in the neighbourhood of a given solution trajectory will *converge* to the given curve. Hence analytical stability is investigated by perturbing the *initial values* in the problem (see section 2.3.1)

Another version of the VE occurs in *sensitivity analysis*, i.e. when we want to compute how sensitive a solution is with respect to *parameters* in the right hand side. Sensitivity analysis is based on the IVP-formulation (3.2)

$$\frac{d\mathbf{u}}{dt} = \mathbf{f}(t, \mathbf{u}, \mathbf{p}), \qquad \mathbf{u}(0) = \mathbf{u}_0(\mathbf{p}) \tag{3.58}$$

To investigate the sensitivity, assume that the solution trajectory $\mathbf{u}(t, \mathbf{p})$ has been computed for a given value of the parameter vector $\mathbf{p}$ and the task is to study a neighbouring solution trajectory $\mathbf{u}(t, \mathbf{p} + \delta\mathbf{p})$. Taylor expansion gives

$$\mathbf{u}(t, \mathbf{p} + \delta\mathbf{p}) = \mathbf{u}(t, \mathbf{p}) + \frac{\partial \mathbf{u}}{\partial \mathbf{p}}(t, \mathbf{p})\delta\mathbf{p} + hot \tag{3.59}$$

Hence we need the *sensitivity matrix* $S(t, \mathbf{p})$ defined as

$$S(t, \mathbf{p}) = \frac{\partial \mathbf{u}}{\partial \mathbf{p}} \tag{3.60}$$

$S(t, \mathbf{p})$ is obtained as the solution of the matrix differential equation derived by differentiating (3.58) with respect to $\mathbf{p}$:

$$\frac{dS}{dt} = \frac{\partial \mathbf{f}}{\partial \mathbf{u}}S + \frac{\partial \mathbf{f}}{\partial \mathbf{p}}, \qquad S(0, \mathbf{p}) = \frac{\partial \mathbf{u}_0}{\partial \mathbf{p}} \tag{3.61}$$

The sensitivity matrix can be used for *parameter estimation*. Assume we want to fit the parameter vector $\mathbf{p}$ in (3.58) to measured data $(t_k, \mathbf{u}_k^*)$, $k = 1, 2, \ldots, M$ of the state variable $\mathbf{u}(\mathbf{t})$. The residual vector at each measurement point is defined as

$$\mathbf{r}_k = \mathbf{u}(t_k, \mathbf{p}) - \mathbf{u}_k^* \approx 0, \quad , k = 1, 2, \ldots, M$$

Also define the global sensitivity matrix $\mathbf{S}(\mathbf{p})$, the global measurement vector

$\mathbf{u}^*$ and the global residual vector $\mathbf{r}(\mathbf{p})$ according to

$$\mathbf{S}(\mathbf{p}) = \begin{pmatrix} S(t_1, \mathbf{p}) \\ S(t_2, \mathbf{p}) \\ . \\ . \\ S(t_M, \mathbf{p}) \end{pmatrix}, \quad \mathbf{u}^* = \begin{pmatrix} \mathbf{u}_1^* \\ \mathbf{u}_2^* \\ . \\ . \\ \mathbf{u}_M^* \end{pmatrix}, \quad \mathbf{r} = \begin{pmatrix} \mathbf{r}_1 \\ \mathbf{r}_2 \\ . \\ . \\ \mathbf{r}_M \end{pmatrix}$$

Gauss-Newton's method can be used (see section A.1). Assume $\mathbf{p}^{(i)}$ is an iterate in a sequence $\mathbf{p}^{(0)}, \mathbf{p}^{(1)}, \ldots, \mathbf{p}^{(i)}$. Given a start parameter vector $\mathbf{p}^{(0)}$, iterate according to

$$\begin{cases} \mathbf{S}(\mathbf{p}^{(i)})\delta\mathbf{p}^{(i)} = -\mathbf{r}^{(i)} \\ \mathbf{p}^{(i+1)} = \mathbf{p}^{(i)} + \delta\mathbf{p}^{(i)} \end{cases}$$

until the correction $\|\delta\mathbf{p}^{(i)}\|$ is small enough.

Example 3.7. Consider the following system of ODEs containing three parameters k_1, k_2 and k_3:

$$\frac{dy_1}{dt} = k_1 y_1 - k_2 y_1 y_2, \quad y_1(0) = 1.0$$

$$\frac{dy_2}{dt} = k_2 y_1 y_2 - k_3 y_2, \quad y_2(0) = 0.3$$

Assume we have measurements of y_1 and y_2 according to the matrix

$$\mathbf{Y}_{msr} = \begin{pmatrix} 1 & 1.1 & 1.3 & 1.1 & 0.9 & 0.7 & 0.5 & 0.6 & 0.7 & 0.8 & 1 \\ 0.3 & 0.35 & 0.4 & 0.5 & 0.5 & 0.4 & 0.3 & 0.25 & 0.25 & 0.3 & 0.35 \end{pmatrix}$$

at time points $t_k = k \cdot 0.5, k = 0, 1, 2, \ldots 10$. We want to estimate the unknown parameters k_1, k_2 and k_3 from the measurements by using the algorithm described in section 3.6. For that we need the following jacobians:

$$J(\mathbf{y}) = \begin{pmatrix} k_1 - k_2 y_2 & -k_2 y_1 \\ k_2 y_2 & k_2 y_1 - k_3 \end{pmatrix}, \quad \frac{\partial \mathbf{f}}{\partial \mathbf{p}} = \begin{pmatrix} y_1 & -y_1 y_2 & 0 \\ 0 & y_1 y_2 & -y_2 \end{pmatrix}$$

The initial guess for the parameter vector is $\mathbf{k}_0 = (1, 1, 1)^T$. After a few iterations with the Gauss-Newton method the iterations have converged (Figure 3.12) to $\mathbf{k}_{est} = (0.86, 2.08, 1.81)$.

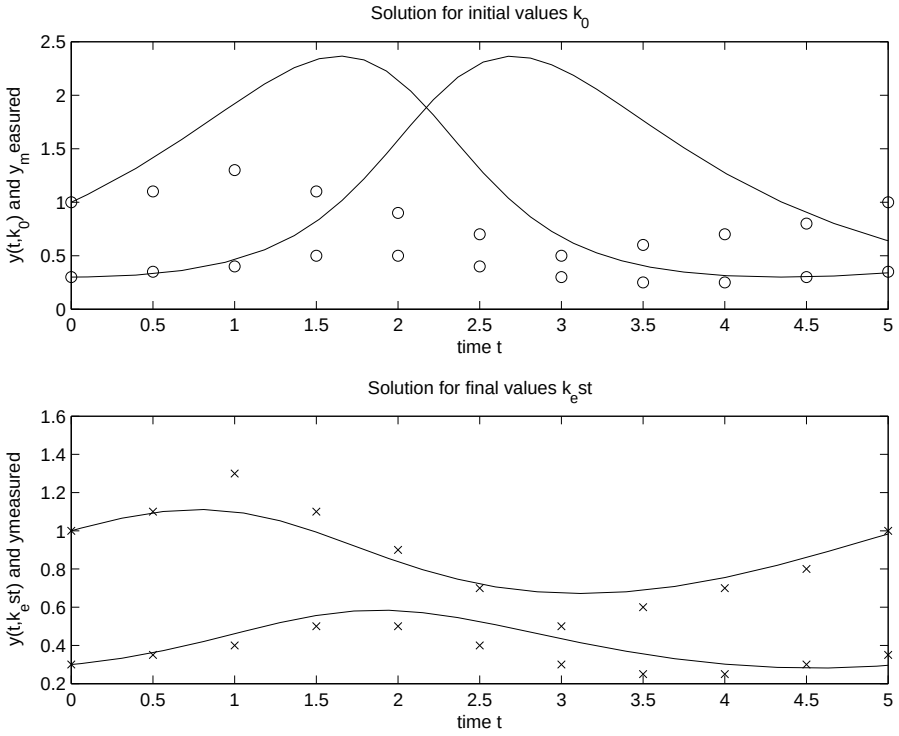


Figure 3.12:Measurements and solution curves before and after the Gauss-Newton method

Bibliography

1) G. Dahlquist, Å. Björck,"Numerical Methods", chapter 8, Dover, 2003

2) G.Golub, J.Ortega,"Scientific Computing and Differential Equations", Chapter 2, Academic Press, 1992

3) C. Moler,"Numerical Computing with MATLAB", chapter 7, SIAM, 2004

A textbook with many physical applications

4) A.L.Garcia, "Numerical Methods for Physics", 2nd ed, chapter 2-3, Prentice Hall, 2000

Extensive treatment of the IVP is found in the textbooks

5) U.Ascher, L.Petzold, "Computer Methods for ODEs and DAEs", SIAM, 1998

6) J.C.Butcher, “Numerical Methods for Ordinary Differential Equations”, Wiley, 2003

7) E.Hairer, S.Nörset, G.Wanner, “Solving Ordinary Differential Equations I: Non-stiff problems”, Springer 2009

8) E.Hairer, G.Wanner, “Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems”, Springer 2010

9) A.Iserles, “A First Course in the Numerical Analysis of Differential Equations”, Cambridge University Press, 1996

10) L.Shampine, ”Numerical Solution of ODEs”, Chapman and Hall, 1994